Name: _____
Surname: _____
Date: _____

Intructions

Always assume each code is run independently, so variables do not persist between exercises.

Questions 1-6: Multiple Choice (5 points each)

Circle the correct answer for each question, as each question has **only one** correct answer. A correct answer awards 5 points. There is no penalty for wrong choices.

Questions 7-9: Debugging (10 points each)

These exercises show code cells with one or more errors. You have to find these errors, explain why they happen, and suggest a fix to them.

Questions 10-11: Open Problems (20 points each)

In these exercises, you will encounter problems that are more open-ended and may not have a single correct answer. Your task is to explain how you would approach each problem by breaking it down into smaller, manageable steps.
Describe how you would structure your code, which logical steps are needed, and what kind of Python tools or constructs (e.g., loops, conditionals, functions, lists, etc.) you would use. You are not required to write full code, but doing so is encouraged if it helps clarify your ideas.

---

1. You want to compute the following expression in Python: $(a + b^2)/(2c)$. Which one of the following options does it right?

   (a) `(a + b∧2) / (2 * c)`

   (b) `(a + b∧2) / 2 * c`

   (c) `(a + b**2) / (2 * c)`

   (d) `(a + b**2) / 2 * c`

2. After running the following code, what will be the final value of variable "s"?

```
s = "aBcDeFg"
s = s[1::3].lower()
```

   (a) `s = "abcdefg"`

   (b) `s = "adg"`

   (c) `s = "be"`

   (d) The code will raise an error.

3. What is the purpose of the “else” statement in an “if-else” block?

   (a) To execute code only if all the above “if” conditions are False.

   (b) To specify an alternative condition.

   (c) To indicate the end of the code block.

   (d) To stop an outer loop.

4. After running the following code, what will be the final value of variable “x”?

```
st_a = {1, 2, 3}
st_b = {2, 2, 3}
st_c = list(st_a.union(st_b))
x = sorted(st_c)[0]
```

   (a) `x = 1`

   (b) `x = 2`

   (c) The code will raise an error.

   (d) We cannot predict the outcome.

5. What is the difference between “print(x)” and “return x” inside an user defined function?

   (a) “print(x)” and “return x” are interchangeable; they both output the value of “x”.

   (b) “print(x)” is a local variable inside the function, while “return x” is a global variable of that function.

   (c) “print(x)” outputs the value of “x” to the console, while “return x” sends the value of “x” back to the calling code.

   (d) “print(x)” will cause an error if we use it inside an user defined function.

6. We run the following code. Which will be the value of “x” in the **global scope**?

```
x = 20

def exercise(y: int) -> int:
  x = 4
  return y / x

x = exercise(x)
```

   (a) `x = 4`

   (b) `x = 5`

   (c) `x = 20`

   (d) The code will raise an error.

Answers: 1-c, 2-c, 3-a, 4-a, 5-c, 6-b

7. This code aims to count the number of unique words in a string, ignoring case sensitivity. However, it doesn't produce the correct count. Find the error and correct it.

```
def count_unique_words(text):
  words = text.lower().split(" ")
  unique_words = set()
  for word in words:
    unique_words.add(word)
  return len(words)

sentence = "Hello world hello"
print(count_unique_words(sentence))  # Output: 3, Expected: 2
```

Answer:

The function builds the set `unique_words` correctly, as a set discards repeated elements. However, the last line returns `len(words)`, which is the length of the list with **all** the words, including the repeated ones. This is why the output is 3. *(+5 points)*

We need to return the length of the set instead, `return len(unique_words)`. In this example, the set is `{"hello", "world"}` and the function returns 2, as expected. *(+5 points)*

(Optional, fixed code)

```
def count_unique_words(text):
    words = text.lower().split(" ")
    unique_words = set()
    for word in words:
        unique_words.add(word)
    return len(unique_words)
```

8. The following function is supposed to merge two dictionaries, without modifying the originals. In case of key conflicts, the value from the second dictionary should overwrite the one from the first. The code raises an error on line 5: "`SyntaxError: cannot assign to function call here`". Identify and fix the issue(s).

```
1  def merge_dictionaries(dict1: dict, dict2: dict) -> dict:
2    merged_dict = dict1
3    for key, value in dict2.items():
4          # Update the key with the value from dict2
5      merged_dict.keys() = value
6    return merged_dict
7
8  dict_a = {"a": 1, "b": 2}
9  dict_b = {"b": 3, "c": 4}
10 print(merge_dictionaries(dict_a, dict_b))
11 # Expected: {"a": 1, "b": 3, "c": 4}
```

Answer:

There are two errors. The first one is in line 5, the one that raises the `SyntaxError`. The method `merged_dict.keys()` returns the keys of the dictionary, and a function call cannot be on the left side of an assignment. To add a pair or replace the value of an existing key, we use the key as an index: `merged_dict[key] = value`. *(+5 points)*

The second error is in line 2. The command `merged_dict = dict1` does not create a new dictionary: both names refer to the **same** dictionary. Once the first error is fixed, every update would also modify `dict1` (which is `dict_a` outside the function), and the exercise asks us not to modify the originals. We need to create a copy with `dict1.copy()` (or `dict(dict1)`). *(+5 points)*

(Optional, fixed code)

```
1  def merge_dictionaries(dict1: dict, dict2: dict) -> dict:
2      merged_dict = dict1.copy()
3      for key, value in dict2.items():
4          # Update the key with the value from dict2
5          merged_dict[key] = value
6      return merged_dict
```

9. This code is meant to generate a list of squares of numbers from 1 to 5 using a while loop. However, it results in an infinite loop. Find the mistake and correct it.

```python
def generate_squares(limit: int=5) -> list[int]:
    squares = []
    i = 1
    while i <= limit:
        squares.append(i ** 2)
    return squares

limit = -1
print(generate_squares())  # Expected: [1, 4, 9, 16, 25]
```

Answer:

The loop condition is `i <= limit`, but the variable `i` never changes inside the loop. It starts at 1 and stays at 1, so the condition is always true and the loop never ends. *(+5 points)*

We need to increase `i` at the end of every iteration, adding `i += 1` **inside** the loop (indented, after the `append`). If it is written outside the loop, it is never reached. *(+5 points)*

Comment: The line `limit = -1` in the global scope is irrelevant: the function uses its own parameter `limit`, whose default value is 5.

(Optional, fixed code)

```python
def generate_squares(limit: int=5) -> list[int]:
    squares = []
    i = 1
    while i <= limit:
        squares.append(i ** 2)
        i += 1
    return squares
```

10. You are given a string of any length, and you want to determine how many times a specific pattern appears in it, ignoring spaces, uppercase letters, and allowing overlaps. For instance, the pattern "ana" appears four times in "Ana ate an apple and a banana". Matches include:

    - "Ana" at the start, ignoring the uppercase.
    - "an-a(pple)" ignoring the space.
    - "(b)ana(na)" and "(ban)ana" overlapping.

    Design an algorithm to perform the task above. It must work for any possible alphanumeric string and for any target pattern.

    Answer:

    The input of the function will be two strings: the main string and the target pattern. The output is an integer, the number of times the pattern appears. *(+2 points)*

    Because the search ignores spaces and uppercase letters, we first clean both strings. We turn them to lowercase with `.lower()`, and we remove their spaces by splitting at the spaces and joining the pieces again with an empty separator: `"".join(text.split(" "))`. We apply the same cleaning to the pattern. *(+5 points)*

    Next, we initialize a counter with `count = 0`, and we loop through every position where a match could start: `for idx in range(len(clean) - len(target) + 1)`. The "+1" is needed to include the last possible starting position. Because we visit every position, including the ones inside a previous match, overlapping matches are counted. *(+5 points)*

    Inside the loop, we take a slice of the same length as the pattern, `clean[idx:idx+len(target)]`, and compare it to the pattern using an if-clause. If they are equal, we add 1 to the counter. After the loop ends, we return the counter. *(+8 points)*

    Comment: Another valid approach is to use `clean.find(target, start)` inside a while loop. After a match found at position `pos`, the next search starts at `pos + 1` (not at `pos + len(target)`), which allows overlaps. The loop ends when `find` returns `-1`. Using `clean.count(target)` is not correct, as it does not count overlapping matches.

    (Optional, code)

```python
def exercise10(text: str, pattern: str) -> int:
    # Clean both strings: lowercase and no spaces
    clean = "".join(text.lower().split(" "))
    target = "".join(pattern.lower().split(" "))
    # An empty pattern has no matches
    if len(target) == 0:
        return 0
    count = 0
    # Look at every possible starting position
    for idx in range(len(clean) - len(target) + 1):
        # Take a slice of the same length as the pattern
        if clean[idx:idx+len(target)] == target:
            count += 1
    return count
```

11. In 2D space, a point can be represented as a pair of coordinates (x, y). The Euclidean distance between two points (x_1, y_1) and (x_2, y_2) is given by the formula:

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

You have:

- A reference point as a tuple of two coordinates: $A = (x_A, y_A)$
- A list of tuples, each representing a point in the 2D space: $[(x_B, y_B), (x_C, y_C), ...]$

Design an algorithm that returns a list of Euclidean distances between the reference point A and each of the other points. That is, return: $[AB, AC, ...]$.

Answer:

The function receives two inputs: the reference point, which is a tuple of two numbers, and the list of points, which is a list of tuples. It returns a list of numbers: one distance for each point, in the same order. *(+3 points)*

Before the loop, we initialize an empty list, `distances = []`, where we will store the results. After the loop finishes, we return it. *(+3 points)*

We need one distance for each point of the list, so we use a for loop, `for point in ls_points`. Each `point` is a tuple, so we can unpack its coordinates with `x2, y2 = point`. We unpack the reference point in the same way, before the loop, with `x1, y1 = point_a`. Indexing, such as `point[0]` and `point[1]`, is equally valid. *(+5 points)*

Inside the loop, we apply the formula. The squares are computed with `**2`. As we have no square root function in the cheatsheet, we use a power of one half, `**0.5`. Be careful with the parentheses, as the exponent must affect the whole sum: `((x2 - x1)**2 + (y2 - y1)**2) ** 0.5`. Using `math.sqrt()` after `import math` is also accepted. *(+5 points)*

Finally, we add each distance to the list, using `distances.append(distance)`. *(+4 points)*

(Optional, code)

```python
def exercise11(point_a: tuple, ls_points: list[tuple]) -> list[float]:
    # Initialize the output list
    distances = []
    # Unpack the coordinates of the reference point
    x1, y1 = point_a
    # Loop through the points, one at a time
    for point in ls_points:
        x2, y2 = point
        # Apply the Euclidean distance formula
        distance = ((x2 - x1)**2 + (y2 - y1)**2) ** 0.5
        distances.append(distance)
    return distances
```

Cheatsheet

Given a number named `x`:

- `x % y` returns the remainder of the division $\frac{x}{y}$.
- `x // y` returns the quotient of the division $\frac{x}{y}$.
- `abs(x)` returns the absolute value of `x`.

Given a string named `s`:

- `int(s)` returns an integer. Fails if `s` has invalid characters.
- `float(s)` returns a floating number. Fails if `s` has invalid characters.
- `s.split(" ")` split a string into a list where each word is a list item.
- `s.count(<pattern>)` returns the number of non-overlapping occurrences of `<pattern>`.
- `s.find(<pattern>, <start>)` finds the first occurrence of the specified `<pattern>`, starting from index `<start>`. Returns `-1` if the pattern is not found.

Given a list or tuple named `ls`:

- `len(ls)` returns the number of elements in that list.
- `sum(ls)` returns the sum of the elements of that list.
- `sorted(ls)` returns a sorted version of that list.
- `tuple(ls)` / `set(ls)` returns a tuple / set using the elements of that list.
- `", ".join(ls)` returns a string with the elements of the list separated by commas.

Given a set named `a`:

- `a.add(x)` adds the element `x` to the set `a`.
- `a.union(b)` returns the union of set `a` with another set `b`.
- `a.intersection(b)` returns the intersection of set `a` with another set `b`.
- `a.difference(b)` returns the difference of set `a` with another set `b`.

Given a dictionary named `d`:

- `list(d.keys())` returns a list with the keys of `d`.
- `list(d.values())` returns a list with the values of `d`.
- `list(d.items())` returns a list with the tuples (key, value) of `d`.

Some useful build-in functions:

- `type(x)` returns the type of `x` (int, float, list...).
- `range(start, stop)` generates numbers from `start` until `stop`, not including `stop`.